

PRÁCTICA DE BLOQUE II APRENDIZAJE AUTOMÁTICO.

Construcción y Almacenamiento de un **Dataset** de Gestión de Pedidos de una Tienda Online

1	Contenido	
2	Objetivos.....	1
3	Herramientas.....	1
4	Fecha clave.....	2
5	Tareas.....	2
5.1	Clasificación con modelos de Machine Learning (4 ptos)	2
5.2	Regresión con modelos de Machine Learning (4 ptos)	2
5.3	Diseñando una red con validación cruzada y regularización (2 ptos)	3

2 Objetivos

- Aplicar técnicas de clasificación supervisada utilizando distintos modelos clásicos como árboles de decisión, máquinas de vectores soporte (SVM) y regresión logística.
- Explorar y ajustar hiperparámetros clave de cada modelo para observar su efecto sobre el rendimiento, fomentando la interpretación de fenómenos como el sobreajuste y el subajuste.
- Entrenar y evaluar una red neuronal simple con TensorFlow/Keras, comprendiendo su estructura, entrenamiento y métricas.
- Comparar de forma cuantitativa y visual los modelos entrenados utilizando:
 - Matrices de confusión
 - Métricas clásicas (Accuracy, Precision, Recall, F1-Score, AUC, Specificity)
 - Curvas ROC
 - Radar charts y barplots para comparar desempeño.
- Extender el análisis a un problema de regresión, aplicando modelos como regresión lineal y redes neuronales, e interpretando métricas como MSE, RMSE, MAE y R^2 .
- Diseñar, entrenar y validar redes neuronales personalizadas en un problema de clasificación más complejo, incorporando técnicas como validación cruzada y regularización mediante Dropout.

3 Herramientas

Durante el desarrollo de esta práctica, se emplearán las siguientes herramientas y librerías del ecosistema Python orientadas al análisis y modelado de datos:

- **Lenguaje de programación Python 3.11.11:** Lenguaje base para todo el desarrollo de la práctica.
- **Librerías para procesamiento de datos:**
 - pandas: Manipulación y análisis de datos estructurados.
 - numpy: Computación numérica eficiente y manejo de arrays.
 - scikit-learn: Implementación de modelos clásicos de machine learning (clasificación, regresión, métricas, validación cruzada).
 - Tensorflow (2.19) + keras: Construcción, entrenamiento y evaluación de redes neuronales.
 - joblib: Serialización y carga de modelos entrenados.
 - Matplotlib y seaborn: Visualización de métricas y resultados (gráficos, matrices de confusión, radar charts, curvas ROC).

4 Fecha clave

- ❖ Fecha de entrega de la práctica (subida a Moodle del documento word): 17/05/2026 a las 23:59.

5 Tareas

5.1 Clasificación con modelos de Machine Learning (4 ptos)

El objetivo de esta sección es explorar y comparar varios modelos de clasificación, tanto tradicionales como una red neuronal simple, para predecir si una persona va a desarrollar una enfermedad cardíaca en base a unas características. Para ello, Utilizaremos el dataset **Heart Disease** (disponible en 'sklearn.datasets'), que contiene características para predicción de enfermedades cardíacas (maligno o benigno). Para ello:

- Primero, cargaremos el dataset y prepararemos los datos
- Entrenaremos y ajustaremos los modelos.
- Reflexionaremos sobre qué modelo tiene el mejor rendimiento para este tipo de clasificación.

Completa además las siguientes preguntas en tu informe:

- ¿Qué observas en los entrenamientos iniciales? ¿Qué le pasa a cada modelo?
- Explica que parámetros influyen en el problema detectado y como has operado para solucionarlo.
- Explica las métricas obtenidas, y compara los modelos entre ellos

Vamos a diseñar ahora un modelo de red neuronal sencillo para este mismo problema de clasificación. Para ello, deberás realizar las siguientes tareas:

- Probar el modelo y explicar las métricas obtenidas
- Cambiar el número de neuronas en las capas ocultas.
- Probar con más o menos capas ocultas.
- Modificar las funciones de activación (por ejemplo, tanh o sigmoidea en las capas ocultas).
- Modificar hiperparámetros para conseguir un accuracy mínimo de 0.85

Una vez que tengas tus modelos optimizados ejecuta `evaluación_modelos_clasificacion.py` e indica lo que observas en relación con la capacidad clasificatoria de cada modelo. Crea una tabla resumen con las métricas.

5.2 Regresión con modelos de Machine Learning (4 ptos)

El código disponible en `Modelos_De_Regresión.py` contiene los modelos vistos en el apartado anterior, aplicados ahora a un problema de regresión. En este caso cargaremos el conjunto de datos de **California Housing**, que representa una variable continua fácilmente aproximable por un modelo de regresión. Realiza un análisis similar al caso anterior, en el que evalúes los modelos y respondas a las siguientes preguntas (Objetivo $R^2 \sim 0.6/07$):

- ¿Qué modelos sobreajustan más fácilmente?

- ¿Qué hiperparámetros resultan más sensibles?

Vamos a evaluar ahora una red neuronal sencilla para regresión, disponible en `Red_Neuronal_Regresion.py`. En este caso, procederemos como en la red de clasificación, tendremos que modificar las neuronas y capas ocultas hasta conseguir un $R^2 \sim 0.6/07$:

- ¿Qué modelo tiene un mejor rendimiento general?
- ¿La red neuronal es más precisa que los modelos clásicos?

Una vez que optimizados todos los modelos, ejecuta `evaluación_modelos_regresion.py` e indica lo que observas en relación con cada modelo.

5.3 Diseñando una red con validación cruzada y regularización (2 ptos)

En esta sección, utilizaremos el conjunto de datos de supervivencia del Titanic disponible en OpenML. Este dataset contiene características de los pasajeros del Titanic (como la clase del billete, el sexo, la edad, etc.) y si el pasajero sobrevivió o no al hundimiento. El objetivo es predecir la supervivencia de un pasajero a partir de sus características.

- **Características (X):** 7 variables (por ejemplo, clase del billete, sexo, edad, número de familiares a bordo, tarifa pagada, puerto de embarque, etc.).
- **Etiquetas (y):** Supervivencia del pasajero (0 = No sobrevivió, 1 = Sobrevivió). Se trata de un problema de clasificación binaria.

Para ello, es necesario diseñar una red neuronal utilizando Keras. El diseño incluirá las siguientes tareas:

- Elegir el número de capas y neuronas.
- Aplicar Dropout en las capas para prevenir el sobreajuste.
- Utilizar función de activación ReLU para las capas ocultas.
- Usar función de activación sigmoide en la capa de salida, ya que estamos realizando clasificación binaria.

Para evaluar el rendimiento del modelo, habrá que utilizar validación cruzada. Keras no soporta directamente la validación cruzada, pero podemos usar `KFold` de `Scikit-learn` para dividir los datos en diferentes subconjuntos de entrenamiento y validación.

El objetivo es que nuestra red neuronal consiga un accuracy mínimo de **0.80**.